Assignment #05 — 15th December, 2025

Instructions for Assignment Submission

1. File Naming and Submission:
   a. Create a ZIP file containing all your solution files.
   b. Name the ZIP file as `assignment05_<ERP>.zip`, where `<ERP>` is your ERP.
   c. Ensure that the ZIP file contains:
      i. C++ source files (one or more `.cpp` / `.h` files)
      ii. Any required input files (e.g., `students.txt`, `courses.txt`, `enrollments.txt`)
      iii. No unnecessary files (executables, temporary files, IDE configs, etc.)
2. Code Neatness:
   a. Ensure your code is well-formatted and easy to read.
   b. Use meaningful variable and function names.
   c. Add comments for non-trivial logic (especially linked list operations and GPA calculation).
3. Additional Instructions:
   a. Do not use STL containers such as `std::list`, `std::map`. All lists must be implemented using pointers and linked lists.
   b. You may use `std::string` and the standard I/O/file streams (`iostream`, `fstream`).
   c. You must use dynamic memory (`new` / `delete`) for nodes and dynamic prerequisite arrays.
4. Plagiarism Policy:
   a. The work submitted must be entirely your own. Any copied code (from classmates, internet, or AI) will result in an automatic zero.

Optional: You may submit via GitHub with a clear `README` for a small bonus.

1 Student Course Registration System

You will build a simplified Student Course Registration System using:

- Structures and the dot operator
- Pointers, dynamic memory, and singly linked lists
- Basic file I/O and operator overloading

There is no waitlist / queue. If a course is full, the enrollment simply fails.

1.1 Data Structures

Use these structures (you may add helper fields if needed, but do not remove any):

Student Node

```
struct Student {
    int student_id;
    std::string name;
    std::string major;
    int total_credits;   // total successfully completed credits
    double gpa;
    Student* next;       // linked list of students
};
```

Course Node

```
struct Course {
    std::string course_code;    // e.g., "CS106B"
    std::string course_name;
    int credits;
    int capacity;               // max students
    int enrolled_count;         // current enrollments
    std::string* prerequisites; // dynamic array of prerequisite course codes
    int prereq_count;           // number of prereqs
    Course* next;               // linked list of courses
};
```

Enrollment Node

```
struct Enrollment {
    int student_id;
    std::string course_code;
    char grade;           // 'A','B','C','D','F','W','P' (pending)
    std::string semester; // e.g., "Fall 2025"
    Enrollment* next;     // linked list of enrollments
};
```

1.2 Student Management

```
// Add new student to system
void add_student(Student*& student_list,
                 int id,
                 const std::string& name,
                 const std::string& major);
```

```
// Find student by ID (nullptr if not found)
Student* find_student(Student* student_list,
                      int student_id);

// Update student major, return true if successful
bool update_major(Student* student_list,
                  int student_id,
                  const std::string& new_major);

// Remove student and all their enrollments
void remove_student(Student*& student_list,
                    Enrollment*& enrollment_list,
                    int student_id);
```

Notes:

- You can insert new students either at the head or tail of the linked list.
- `remove_student` must delete:
  - the corresponding `Student` node, and
  - all `Enrollment` nodes for that student.

1.3 Course Management (Dynamic Array of Prerequisites)

```
// Add new course to catalog
void add_course(Course*& course_list,
                const std::string& code,
                const std::string& name,
                int credits,
                int capacity);

// Find course by course code
Course* find_course(Course* course_list,
                    const std::string& course_code);

// Add a prerequisite code to an existing course
void add_prerequisite(Course* course,
                      const std::string& prereq_code);

// Remove course (only if no one is enrolled in it)
bool remove_course(Course*& course_list,
                   Enrollment* enrollment_list,
                   const std::string& course_code);
```

Dynamic prerequisites:

- `add_prerequisite` should:
  1. Allocate a new array of size `prereq_count + 1`.
  2. Copy existing prerequisite codes.
  3. Add the new code at the end.
  4. Delete the old array using `delete[]` and update the pointer.
- `remove_course` returns `false` if there is any enrollment node with that course code.

1.4 Enrollment Management (No Waitlist)

```
// Has student already taken / enrolled in this course?
bool already_enrolled(Enrollment* enrollment_list,
                      int student_id,
                      const std::string& course_code);
```

```
// Check if student meets prerequisites (grade C or better)
bool check_prerequisites(Enrollment* enrollment_list,
                         Course* course,
                         int student_id);

// Check credit hour limit (max 18 for given semester)
bool check_credit_limit(Enrollment* enrollment_list,
                        Course* course_list,
                        int student_id,
                        const std::string& semester,
                        int new_course_credits);

// Enroll student in course with validation (no waitlist)
bool enroll_student(Student* student_list,
                    Course* course_list,
                    Enrollment*& enrollment_list,
                    int student_id,
                    const std::string& course_code,
                    const std::string& semester);

// Drop course
bool drop_course(Enrollment*& enrollment_list,
                 Course* course_list,
                 int student_id,
                 const std::string& course_code);

// Assign grade to an enrollment
bool assign_grade(Enrollment* enrollment_list,
                  int student_id,
                  const std::string& course_code,
                  char grade);
```

Validation Rules for **`enroll_student`**

When enrolling, you must check in this order:

1. Student exists: `find_student` is not `nullptr`.
2. Course exists: `find_course` is not `nullptr`.
3. Not already enrolled: `already_enrolled` returns `false`.
4. Prerequisites met: `check_prerequisites` returns `true`.
5. Credit limit: `check_credit_limit` must ensure total credits ≤ 18 for that semester.
6. Capacity: if `enrolled_count >= capacity`, the function returns `false` (enrollment fails).

If all checks pass:

- Insert a new `Enrollment` node.
- Set grade to ''P'' (pending).
- Increase the course's `enrolled_count`.

1.5 GPA Calculation

```
// Update student's GPA based on all enrollments
void update_gpa(Student* student_list,
                Enrollment* enrollment_list,
                int student_id);
```

Use the following grading scheme:

```
Grade Points:
  A = 4.0
  B = 3.0
  C = 2.0
  D = 1.0
  F = 0.0
  W = Withdraw  (not counted)
  P = Pending   (not counted)

GPA = (sum of (grade_points * course_credits)) / (total credits attempted)
Only A,B,C,D,F are counted in numerator and denominator.
```

Also update `total_credits` to reflect total successfully completed credits (e.g., grade C or above).

1.6 Reporting Functions

```
// List all courses a student is enrolled in for a given semester
void list_student_courses(Enrollment* enrollment_list,
                          Course* course_list,
                          int student_id,
                          const std::string& semester);

// List all students currently enrolled in a given course
void list_course_students(Enrollment* enrollment_list,
                          Student* student_list,
                          const std::string& course_code);

// Generate a simple transcript for a student
void generate_transcript(Student* student_list,
                         Enrollment* enrollment_list,
                         Course* course_list,
                         int student_id);
```

Output can be simple text tables; it does not need to match the long stylized sample in the earlier description.

1.7 Operator Overloading

To practice operator overloading, define:

```
// Pretty-print basic student information
std::ostream& operator<<(std::ostream& out, const Student& s);
```

Example output:

```
ID: 12345, Name: Ali Memon, Major: Computer Science, Credits: 45, GPA: 3.67
```

Use this in your menu options when displaying students.

1.8 File I/O (Simplified Core)

```
// Save all lists to files
void save_system(Student* student_list,
                 Course* course_list,
                 Enrollment* enrollment_list);

// Load lists from files (overwrite existing in-memory data)
void load_system(Student*& student_list,
                 Course*& course_list,
                 Enrollment*& enrollment_list);
```

You may use simple CSV-like formats such as:
`students.txt`:

```
12345,Ali Memon,Computer Science,45,3.67
12346,Ammar,Software Engineering,30,3.25
```

`courses.txt`:

```
CSE141A,Introduction to Programming,5,90,
CSE142,Object Oriented Programming Techniques,5,90,CSE141A
CSE143,Design and Analysis of Algorithms,5,90,CSE141A;CSE142
```

`enrollments.txt`:

```
12345,CSE141A,A,Fall 2024
12345,CSE142,P,Spring 2025
12346,CSE143,C,Fall 2024
```

You can parse these using `std::getline` and manual splitting.

1.9 Main Menu and Testing

Your `main()` function should provide at least the following menu:

```
=== University Course Registration System ===

1. Load data from files
2. Save data to files
3. Add student
4. Add course
5. Enroll student in course
6. Drop course
7. Assign grade
8. List student courses (by semester)
9. List course students
10. Generate transcript
11. Remove student
12. Exit
```

This menu is mainly for testing your functions; a text-based interface is enough.

1.10 Testing Scenarios

You should test at least:

1. Successful enrollment: all checks pass.
2. Prerequisite not met: enrollment fails.
3. Credit limit exceeded: student already has 15–18 credits, adding another 4–5 should fail.
4. Course full: `enrolled_count == capacity`, further enrollments fail.
5. GPA update: after assigning grades, GPA reflects correct value.
6. Removal: removing a student also removes all their enrollments.
7. File I/O: save, restart program, load, and verify that data matches.

1.11 Common Mistakes to Avoid

1. Memory Leaks: Forgetting to `delete` nodes (or prerequisite arrays) when removing students/courses/enrollments.
2. Dangling Pointers: Calling `delete` on a node but still using the old pointer, or not setting it to `nullptr` when appropriate.

3. Lost Head Pointer: Modifying the head of a linked list inside a function without using a reference parameter (e.g., `Student* & head`).
4. Infinite Loops: Forgetting to advance the traversal pointer inside a `while (current != nullptr)` loop.
5. Duplicate IDs: Not checking for an existing `student_id` or `course_code` before adding a new node.
6. Division by Zero: Computing GPA when the total attempted credits are zero.
7. File Format Issues: Not trimming spaces or handling newlines properly when parsing CSV-style files.
8. Grade Validation: Accepting invalid grade characters instead of restricting to `'A','B','C','D','F','W'`.
9. Null Pointer Dereference: Not checking if pointers are `nullptr` before accessing their fields.
10. Enrollment Count Bugs: Forgetting to update `enrolled_count` when enrolling or dropping a student from a course.

1.12 Grading Breakdown

- Data Structures & Linked Lists (30%):
  - Correct use of `struct` and linked lists for students, courses, enrollments.
  - Proper insertion, search, and deletion.
- Dynamic Memory & Pointers (25%):
  - Correct use of `new` / `delete` and dynamic arrays for prerequisites.
  - No obvious memory leaks in normal usage.
- Core Functionality (25%):
  - Enrollment validation, credit limit, capacity checks.
  - GPA computation and updating of `total_credits`.
  - Reporting functions produce sensible output.
- File I/O & Menu (10%):
  - System can be saved and loaded from files.
  - Menu allows basic interaction and testing.
- Code Quality & Style (10%):
  - Clear structure, meaningful names, comments on tricky parts.
  - Reasonable modularization into functions.

Good luck, and have fun building your Student Course Registration System!